

Advanced Parallel Grid Processing with MPI

Project Documentation

a. Design Overview

System Architecture

The system is a parallel distributed processing framework built with MPI in C++. It supports two algorithms selectable at runtime: **Game of Life** (Category A – Grid/Spatial) and **Matrix Multiplication** (Category B – Data/Computation). All input and output is handled exclusively by rank 0, while computation is distributed across all available processes. The menu loop keeps the program running until the user explicitly chooses Quit, allowing multiple algorithm runs without restarting.

File Structure

File	Role
main.cpp	Entry point, menu loop, input validation, I/O coordination
config.h	Shared data structures: Matrix , IntMatrix , AlgorithmType
io_utils.h / io_utils.cpp	File reading and writing for double matrices and integer patterns
mpi_utils.h / mpi_utils.cpp	All parallel algorithm implementations and MPI communication logic

Runtime Flow

Program Start

|



Main Menu Loop

|

— 1. Game of Life

|

— Read pattern file (.txt) on rank 0

|

— Broadcast rows/cols/iterations

|

— Broadcast full grid (MPI_Bcast)

|

— ScatterV rows to all processes

|

— Iterate:

|

Non-blocking ghost row exchange

|

Compute interior rows (overlap)

|

MPI_Waitall

|

Compute boundary rows

|

GatherV + print iteration to terminal

|

MPI_Barrier

|

— 2. Matrix Multiplication

|

— Read Matrix A and B from files

|

— Broadcast full B to all workers

|

— Coordinator dispatches row chunks

|

— Workers compute local C and return

|

— Coordinator assembles full result C

|

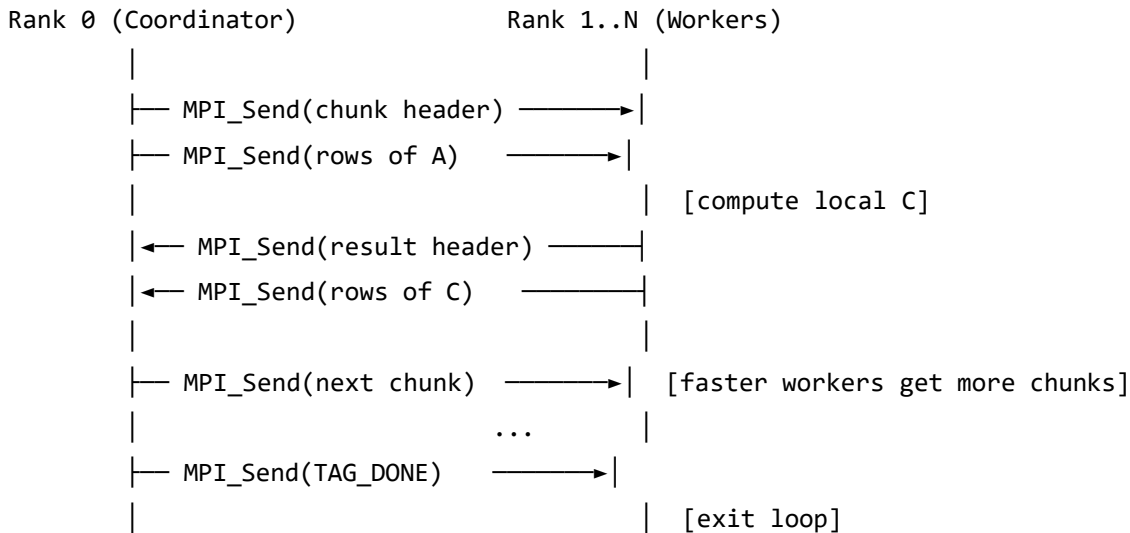
— 3. Quit

b. Communication Strategies

The system implements three distinct communication strategies as required by the project specification.

Strategy 1 — Blocking Communication (MPI_Send / MPI_Recv)

Used in the **Matrix Multiplication Master-Worker** pattern.



```

// Coordinator sending chunk – BLOCKING
MPI_Send(header, 2, MPI_INT, w, TAG_WORK, comm);
MPI_Send(A + nextRow * K, chunkSize * K, MPI_DOUBLE, w, TAG_WORK, comm);

// Coordinator receiving result – BLOCKING
MPI_Recv(resultHeader, 2, MPI_INT, MPI_ANY_SOURCE, TAG_RESULT, comm, &status);
MPI_Recv(C + startRow * M, chunkSize * M, MPI_DOUBLE, workerRank, TAG_RESULT, comm, ...)

// Workers receiving chunk – BLOCKING
MPI_Recv(header, 2, MPI_INT, 0, MPI_ANY_TAG, comm, &status);

// Workers sending result – BLOCKING
MPI_Send(resultHeader, 2, MPI_INT, 0, TAG_RESULT, comm);
MPI_Send(localC.data(), chunkSize * M, MPI_DOUBLE, 0, TAG_RESULT, comm);
  
```

This works without deadlock because the coordinator and workers always perform opposite operations — when the coordinator sends, the worker receives, and vice versa. There is no circular wait.

This pattern also implements **Advanced Load Balancing**: faster processes complete their chunk and immediately receive a new one, so no process sits idle while others are still

computing.

Strategy 2 — Non-Blocking Communication (MPI_Isend / MPI_Irecv)

Used in the **Game of Life ghost row exchange** every iteration.

Each process owns a horizontal strip of the grid and needs one row from the process above and one from below (ghost rows) to compute its boundary cells.

```

Process 0 | ghost      | ← Irecv — | real row 1 | Process 1
          | real row 0 | — Isend → | ghost      |
          | real row 1 | — Isend → | ghost      |
          | ghost      | ← Irecv — | real row 0 |

```

```

// Post all 4 messages simultaneously – none block
MPI_Isend(&localGrid[1 * cols], cols, MPI_INT, upRank, TAG_DOWN, comm, &reqs[0]);
MPI_Irecv(&localGrid[0], cols, MPI_INT, upRank, TAG_UP, comm, &reqs[1]);
MPI_Isend(&localGrid[localRows * cols], cols, MPI_INT, downRank, TAG_UP, comm, &reqs[2]);
MPI_Irecv(&localGrid[(localRows+1)*cols], cols, MPI_INT, downRank, TAG_DOWN, comm, &reqs[3]);

// Compute INTERIOR rows while ghost rows travel (overlap) for
for (int i = 2; i <= localRows - 1; i++) { ... }

// Wait only when boundary rows are needed
MPI_Waitall(4, reqs, MPI_STATUSES_IGNORE);

// Now safe to compute BOUNDARY rows that need ghost data
for (int i : {1, localRows}) { ... }

```

The key benefit is **computation-communication overlap** — interior rows are computed while ghost rows are still in transit, hiding communication latency.

Strategy 3 — Collective Communication (MPI_Scatterv / MPI_Gatherv / MPI_Bcast)

Used in both algorithms for data distribution and collection.

Collective	Used In	Purpose
MPI_Bcast	Both algorithms	Broadcasts full B matrix and initial GOL grid from rank 0 to all processes
MPI_Scatterv	Game of Life	Distributes grid rows unevenly — the V variant handles non-divisible row counts

MPI_Gatherv	Game of Life	Collects result rows back to rank 0 after each iteration for terminal printing
-------------	--------------	--

```
// ScatterV handles uneven row distribution automatically
// e.g. 10 rows across 3 processes: ranks get 4, 3, 3 rows
MPI_Scatterv(
    rank == 0 ? initialGrid.data() : nullptr,
    sendcounts.data(), displs.data(), MPI_INT,
    &localGrid[1 * cols], // skip ghost row 0
    localRows * cols,
    MPI_INT, 0, comm
);
```

Strategy 4 — Process Organization (MPI_Comm_split)

Used in both algorithms to divide processes into logical groups with their own communicators.

Algorithm	Split Criteria	Group 0	Group 1
Matrix Multiplication	rank == 0 ? 0 : 1	Coordinator (rank 0 only)	Worker processes (rank 1..N)
Game of Life	rank % 2	Even-ranked processes	Odd-ranked processes

```
// Matrix Multiplication – coordinator vs workers
int color = (rank == 0) ? 0 : 1;
MPI_Comm group_comm;
MPI_Comm_split(comm, color, rank, &group_comm);

// Game of Life – parity-based split
int color = rank % 2;
MPI_Comm parity_comm;
MPI_Comm_split(comm, color, rank, &parity_comm);
```

c. Deadlock Explanation

What is a Deadlock?

A deadlock occurs when two or more processes are each waiting for the other to act, and none of them can ever proceed. In MPI, the most common cause is all processes calling `MPI_Send` at the same time — because `MPI_Send` blocks until the receiver is ready, but if everyone is sending, nobody is receiving.

Real Deadlock Scenario — Ghost Row Exchange

In Game of Life, every process needs to exchange boundary rows with its neighbors at the start of each iteration. If done with blocking `MPI_Send`, this creates a deadlock..

The Deadlocked Version (wrong implementation):

```
MPI_Send(&localGrid[1 * cols], cols, MPI_INT, upRank, TAG_DOWN, comm); // BLOCKS

MPI_Recv(&localGrid[0], cols, MPI_INT, upRank, TAG_UP, comm, &status);

// NEVER REACHED
MPI_Send(&localGrid[localRows * cols], cols, MPI_INT, downRank, TAG_UP, comm);

// NEVER REACHED

MPI_Recv(&localGrid[(localRows+1)*cols], cols, MPI_INT, downRank, TAG_DOWN, comm,
&status); // NEVER REACHED
```

Why It Happens

Think of 4 processes standing in a circle, each holding a letter they want to pass to the person on their left. But the rule is: *you cannot pass your letter until the person on your right takes yours*. Everyone is waiting for their neighbor to take their letter first — so nobody moves, forever.

```
Process 0  sending to Process 1 → waiting for Process 1  to receive
Process 1  sending to Process 2 → waiting for Process 2  to receive
Process 2  sending to Process 3 → waiting for Process 3  to receive
Process 3  sending to Process 0 → waiting for Process 0  to receive

      ↑_____|
      nobody ever receives
```

→ DEADLOCK

When It Happens

This deadlock would occur at the start of **every iteration** of the Game of Life loop, during the ghost row exchange phase, when all processes simultaneously try to send their boundary rows to neighbors before posting any receives.

The Fix — Non-Blocking Communication

Correct Implementation:

```
MPI_Isend(&localGrid[1 * cols],cols, MPI_INT, upRank, TAG_DOWN, comm, &reqs[0]);
```

```
MPI_Irecv(&localGrid[0],cols, MPI_INT, upRank, TAG_UP, comm, &reqs[1]);
```

```
MPI_Isend(&localGrid[localRows * cols],cols, MPI_INT, downRank, TAG_UP, comm, &reqs[2]);
```

```
MPI_Irecv(&localGrid[(localRows+1)*cols], cols, MPI_INT, downRank, TAG_DOWN, comm, &reqs[3]);
```

MPI_Isend and MPI_Irecv return **immediately** without waiting. All messages are posted at once and the MPI runtime delivers them in the background. No process is ever stuck waiting – so the circular wait is impossible..

d. Performance Observations

Game of Life Results

Pattern: *[grid500.txt]*

Iterations: *[100]*

Processes	Execution Time (sec)
2	<i>[3.00567]</i>
4	<i>[2.17614]</i>
8	<i>[1.84688]</i>
16	<i>[2.11823]</i>

Speedup (relative to 2 processes):

2	1.00 (baseline)
4	<i>1.381193305577766</i>
8	<i>1.627431127090011</i>
16	<i>1.418953560283822</i>

Matrix Multiplication Results

Matrix size: 1000 × 1000

Processes**Execution Time (sec)**

2	[43.7894]
4	[16.059]
8	[10.6628]
16	[8.95472]

Speedup (relative to 2 processes):**Processes****Speedup (T_2 / T_n)**

2	1.00 (baseline)
4	2.726782489569712
8	4.106744945042578
16	4.890091482480748

```
C:\Windows\System32\cmd.exe X + v

C:\Users\HFCS\OneDrive\Desktop\Parallel Computing\Parallel_Project\Debug>mpiexec -n 8 Parallel_Project.exe

=====
MPI Parallel Processing System
Running on 8 processes
=====
Choose Algorithm:
1. Game of Life
2. Matrix Multiplication
3. Quit
> 2
Enter file name for Matrix A:
> e.txt
Matrix A loaded: 1000x1000
Enter file name for Matrix B:
> f.txt
Matrix B loaded: 1000x1000
A: 1000x1000, B: 1000x1000
[Coordinator] Master-Worker Matrix Multiply
[Coordinator] Workers: 7, Chunk size: 35 rows
[Coordinator] All chunks processed
Execution Time: 10.0623 seconds
Enter output file name:
> output.txt
Result written to output.txt

=====
MPI Parallel Processing System
Running on 8 processes
=====
Choose Algorithm:
1. Game of Life
2. Matrix Multiplication
3. Quit
> |
```

```
C:\Windows\System32\cmd.exe X + v

Choose Algorithm:
1. Game of Life
2. Matrix Multiplication
3. Quit
> 1
Enter pattern file name (e.g. glider.txt):
> glider.txt
Pattern loaded: 10x15 grid
Enter number of iterations (must be > 0):
> 2

=== Initial State (Iteration 0) ===
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

=== Final State (after 2 iterations) ===
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Total Execution Time: 0.0012998 seconds
Enter output file name for final grid:
> output2.txt
Final grid written to output2.txt
```



```
C:\Windows\System32\cmd.e x + v
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 1 1 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

=== Final State (after 2 iterations) ===
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 1 0 1 0 0 0 0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Total Execution Time: 0.0012998 seconds
Enter output file name for final grid:
> output2.txt
Final grid written to output2.txt

=====
MPI Parallel Processing System
Running on 8 processes
=====
Choose Algorithm:
1. Game of Life
2. Matrix Multiplication
3. Quit
> 3

[INFO] Goodbye!
C:\Users\HFCS\OneDrive\Desktop\Parallel Computing\Parallel_Project\Debug>
```